

RobotPU Soccer Final Project

Contents

Chapter 1: Idea and System Overview	2
1. Game rules	3
2. System architecture (what runs where)	4
3. Coordinate frames and transforms (high level)	5
3.1 Physical Coordinates (Right-Hand Rule)	5
3.2 Transformations	5
4. Odometry	6
5. MVP-first implementation order (dependency order)	6
6. Practical constraints (micro:bit)	6
7. How to program RobotPU with MakeCode JavaScript	6
7.2 Development Tutorials	7
8. References	11
Chapter 2: Robot PU Odometry	11
Study case 1: one step	13
Study case 2: two steps	13
Study case 3: five steps	14
Study case 4: turn	15
Chapter 3: Local Mapping and Latency Compensation on Robot PU	21
Quick overview (what you are building)	22
Feasibility notes (micro:bit V2 limits)	22
Minimal viable pipeline (dependency order)	22
Game rules (10x10 grid setup)	23
1. System Architecture & Physical Configurations	23
2. Mathematical Foundation	26
3. Implementation roadmap (MVP first)	31
4. Calibration & Testing Protocol	35
Chapter 4: Path Planning and Following	35
Follow Viewpoints	36
Advanced background: Divergent Component of Motion (DCM) tra- jectory generation	40

The Execution Engine: Model Predictive Control (MPC) with a Re- ceding Horizon	40
How to Practicalize It (The Software Pipeline)	41
Chapter 5: Sensor Fusion	42
1. What are we estimating?	42
2. Fusion pattern: prediction + correction	42
3. Camera measurement robustness (must-have)	43
4. Micro:bit fusion recipe (recommended)	43
5. Sonar fusion for mapping	43
6. Kalman filter role in this fusion	43
7. How the code maps to the math	45
8. Gating and dropout behavior	45
9. Practical tuning tips (micro:bit scale)	46
10. Checklist and test protocol	46
Chapter 6: Soccer MVP	46
1. Inputs and outputs	47
2. Coordinate and frame conventions (what the demo actually uses) .	47
3. End-to-end pipeline in <code>robotpu-soccer-demo.ts</code>	47
4. Planning: A* on the 10x10 local grid	48
5. Control: heading-aware approach, then align + kick	49
6. Search behavior	49
7. Known limitations (alpha)	49
8. Suggested testing protocol	50
Chapter 7: Behavior Tree Planner	50
Tree structure	51
Full TypeScript implementation	51
How to integrate with <code>robotpu-soccer-mvp.ts</code>	57
Extending the tree	57
Tick trace example	57

Chapter 1: Idea and System Overview

Welcome to Robot PU soccer project. In this project, you will practice SLAM to build a robot that can navigate to a ball, push it into the goal, and play a simple soccer game.

This final project combines:

- **micro:bit V2** as the main controller (navigation + mapping + state machine)
- **ESP32-S3 camera** for vision (ball/goal detection + 3D estimate in camera frame)
- **sonar** for obstacle detection (short-range safety layer)

- **servos** for walking, head movement, and camera pose estimation.

Because the micro:bit has tight RAM limits, the design intentionally offloads heavy vision to the ESP32 and keeps the micro:bit focused on a **small** 10×10 **grid** and simple closed-loop behaviors.

This document is the *idea + architecture* chapter. Implementation details are in:

- 02_odometry.md (RobotPU walking odometry and the SE(2) update model)
 - 03_localmap.md (I2C packet handling, coordinate transforms, grid mapping)
 - 04_path.md (path planning and following)
 - 05_sensor_fusion.md (Kalman Filter for sensor fusion)
 - 06_soccer_mvp.md (minimal viable product with simple decision making)
 - 07_soccer_bt.md (behavior tree planner, end-to-end demo of the project)
-

1. Game rules

We use a 10×10 grid as a simplified soccer field (1 m x 1 m, so each cell is about 10 cm x 10 cm). Grid coordinates are (x, y) with each index in $[0, 9]$.

- **Goal center location:** $(0, 5)$ with size of A4 paper (210 mm height x 297 mm width)
- **Ball:** 5cm diameter with color (255,98,81) in RGB color.
- **Ball placement:** random mid-field cell
- **Robot placement:** random on the 10th row: $(9, y)$, $y \in [0, 9]$
- **Initial visibility:** robot starts facing so ball and goal are in camera view
- **Objective:** navigate to a good kick position, then push the ball into the goal

1.1 Suggested Grading Criteria

Each feature contributes to a specific grade level:

- Follow ball +1
- Navigate to kick position +1
- Contact ball with goal orientation +1
- Push ball toward goal +1
- Score +1
- Obstacle avoidance +1
- Multiple robot interactions +1

Score of 5 is full marks for A grade. Score above 5 is A+ grade.

Examples of what each grade level could achieve:

- B-, detect and follow the ball

- B, compute and navigate to the kick position behind the ball
- B+, reach the kick position and orient toward the goal
- A-, contact the ball and push or kick it toward the goal
- A, score by pushing or kicking the ball into the goal
- A+, score with obstacle avoidance, or show multiple robot interactions.

1.2 Grading Rubric

- Give a live demo, and submit a video of the robot soccer demo.
 - Submit the *.hex file to the project folder.
 - Submit source code with meaningful comments to the project folder.
 - Submit a README.md file to the project folder, with code references about how mapping, localization, navigation, and control are implemented.
-

2. System architecture (what runs where)

Split responsibilities to fit the micro:bit constraints.

2.1 ESP32-S3 (vision coprocessor)

- Detect ball/goal in the image.
- Produce a compact I2C packet (including a **VALID**/**STALE** flag).
- Provide an estimated (x, y, z) of the ball (in the **camera frame**, units typically mm).

If you want to debug while viewing the image, you can use the Wi-Fi SSID “RobotPU-*****” to connect to the camera server. The password is “robot1234”. Then, use a web browser to access <https://192.168.4.1>. From the camera web page, you can view detected objects:

- Soccer balls in colored bounding box.
- Soccer goals in colored bounding box.
- Faces in green bounding box.

You can also adjust the camera parameters in the code:

- Detection ON/OFF for face, ball, goal.
- Soccer ball color detection (RGB).
- Soccer goal color detection (RGB).
- Soccer ball size detection (diameter).
- Soccer goal size detection (width).

2.2 micro:bit (state + mapping + control)

- Read the I2C packet.
- Read current head yaw/pitch (servo targets).

- Convert camera-frame (x, y, z) into **robot-base coordinates** using head angles + camera mounting offset.
 - Maintain:
 - A small occupancy grid for **obstacles** (sonar)
 - Separate state/layers for **ball** and **goal** (so they don't get "baked" into obstacles)
 - Compute a kick target behind the ball and navigate to it.
-

3. Coordinate frames and transforms (high level)

3.1 Physical Coordinates (Right-Hand Rule)

We strictly adhere to the standard 3D **Right-Hand Rule (RHR)** coordinate system relative to the **Robot Body Base Frame (Origin: 0,0,0 ground center)**:

- **X-Axis:** Horizontal plane, pointing **Left (-)** and **Right (+)**.
- **Y-Axis:** Horizontal plane, pointing **Forward (+)** and **Backward (-)**.
- **Z-Axis:** Vertical axis, pointing **Up (+)** and **Down (-)**.

Same as 3D printer coordinate system.

Angle convention used in the remaining chapters (standard robotics/math convention in the XY plane):

- $\theta = 0$ **points along +X**
- **positive θ rotates CCW toward +Y**
- bearing to a point is `atan2(dy, dx)`

Practical RobotPU note:

- `robotPu.locationArray()[2]` uses a +Y-forward convention
 - 0 means robot facing +Y, moving forward straight.
 - positive means robot turning left.
 - negative means robot turning right.
- When using standard math formulas, convert to standard heading with care.
- The exact conversion used by the odometry and demo code is explained in `02_odometry.md` and `06_soccer_mvp.md`.

3.2 Transformations

You will be using the same story consistently across the project:

- The ESP32 gives ball position in the **camera frame**.
- The robot head can yaw/pitch, so you must rotate the point into the robot base frame.
- Then apply a fixed camera mounting offset (rigid translation).

- Finally, project to a 2D ground plane for grid navigation.

The full math and a practical “recipe” are in `03_localmap.md`.

4. Odometry

RobotPU walking produces motion even if the wheels/feet don’t have encoders. In this project we use a toy but useful model:

- Each step is a rotation about the current support foot pivot (± 25 mm, 0) in the robot frame.
- We represent pose as an SE(2) matrix $T_{world \leftarrow robot}$.
- Each step produces an SE(2) increment T_{step} .
- Odometry updates are composed as:

$$T_{k+1} = T_k T_{step}$$

Details + verified study cases are in `02_odometry.md`.

5. MVP-first implementation order (dependency order)

Do the project in this order so each part provides the inputs for the next:

1. **I2C read and packet validity (VALID/STALE)**
2. **Ball position in robot frame** (head yaw/pitch + camera offset)
3. **2D projection + mapping into the 10×10 grid**
4. **Kick target computation** (point behind ball along ball-to-goal line)
5. **Navigation** (simple steering first, then obstacle avoidance)

This exact dependency order is expanded in `03_localmap.md`.

6. Practical constraints (micro:bit)

- **Keep arrays small and static** (10x10 grids are fine).
- **Avoid allocating new objects** inside `basic.forever`.
- **Prefer integer / fixed-point math** for grid logic when possible.
- **Validate algorithms in Python first**, then port to MakeCode TypeScript/JavaScript.

7. How to program RobotPU with MakeCode JavaScript

To implement this project, you need to use the RobotPU JavaScript API. The API is documented in the RobotPU JavaScript Documentation.

7.1 Milestone

The milestone of this project is:

- ☐ Set up the development environment, and able to run the barebone program to remote control the robot
- ☐ Implement the robot odometry tracking using robot PU API
- ☐ Implement the soccer ball/goal detection and following.
- ☐ Implement the local map building and localization.
- ☐ Implement the navigation to the ball and kick.
- ☐ Implement SLAM-inspired local mapping and localization to make the robot push the ball into the goal.

7.2 Development Tutorials

This project is implemented using **MakeCode JavaScript (TypeScript)** with the `pxt-robotpu` extension.

7.2.1 Setup (barebone program)

Follow the RobotPU barebone tutorial:

- <https://github.com/robotgyms/pxt-robotpu/blob/main/tutorials/JavaScripts/barebone.md>

Minimal workflow:

1. Open MakeCode for micro:bit in your browser.
2. Create a new JavaScript project.
3. Add the RobotPU extension (`pxt-robotpu`).
4. Connect micro:bit V2 over USB.
5. Flash the program.
6. Confirm you can run a minimal program that:
 - sets RobotPU radio channel,
 - receives radio commands (`radio.onReceivedString`, `radio.onReceivedValue`),
 - issues basic RobotPU API calls.

Uploading a downloaded `.hex` file to micro:bit:

1. Connect the micro:bit V2 to your computer with a USB data cable.
2. Wait until the micro:bit appears as a removable drive named MICROBIT.
3. In MakeCode, click **Download** to generate the `.hex` file.
4. Drag the downloaded `.hex` file onto the MICROBIT drive.
5. Wait until the yellow LED stops flashing and the MICROBIT drive reconnects.
6. Press the micro:bit reset button if the program does not start automatically.

This verifies that your dev environment, flashing, and RobotPU runtime are working.

7.2.2 Odometry (read RobotPU pose)

Follow the RobotPU odometry tutorial:

- <https://github.com/robotgyms/pxt-robotpu/blob/main/tutorials/JavaScripts/slam-odometry-pu.md>

In this project you will use odometry primarily to:

- read the robot pose array (e.g. `robotPu.locationArray()`),
- compensate camera latency by transforming detections from the image time to the current time.

7.2.3 Soccer ball following (vision + closed loop control)

Follow the soccer ball follow tutorial:

- <https://github.com/robotgyms/pxt-robotpu/blob/main/tutorials/JavaScripts/slam-soccer-ball-follow.md>

For this final project, you will reuse the same structure:

- enable the ESP32 vision services (ball/goal detection),
- poll I2C packets from the ESP32,
- gate behavior on packet status (e.g. `VALID/STALE`),
- implement a simple head-lock + walking controller.

7.2.4 Build Local Map

Read the local mapping chapter:

- `03_localmap.md`

The goal of this step is to convert camera detections and odometry into a small robot-local soccer field model that can run on micro:bit V2.

For the MVP, keep the map simple:

- use a 10x10 grid for the field,
- treat `x` as left/right and `y` as forward,
- store ball, goal, kick point, and optional obstacle cells as separate layers,
- keep arrays small and reuse them inside `basic.forever`.

Implementation reference:

- `robotpu-localmap.ts`
- `robotpu-map.ts`

Recommended workflow:

1. Read a ball or goal detection from the ESP32 camera packet.
2. Interpret the packet position as a 2D point in the camera/robot-local frame.
3. Use odometry to compensate camera latency.

4. Convert the compensated 2D point into a grid cell.
5. Mark the ball, goal, and computed kick point in the grid.

7.2.5 Path Planning

Read the path planning chapter:

- `04_path.md`

The MVP navigation problem is not full global planning. The robot only needs to move from its current local pose to useful soccer viewpoints:

- approach the kick point behind the ball,
- face the goal,
- push or kick the ball toward the goal.

Implementation reference:

- `robotpu-A-star.ts`
- `robotpu-viewpoint.ts`
- `robotpu-pure-pursuit.ts`

Recommended workflow:

1. Compute a kick point behind the ball along the goal-to-ball direction.
2. Plan a short path on the 10x10 grid using A* if obstacles are present.
3. Convert grid cells into local target poses or viewpoints.
4. Use a viewpoint controller to generate `walkSpeed` and `walkTurn`.
5. Stop when the robot is close enough to the kick point and aligned with the goal.

7.2.6 Sensor Fusion

Read the sensor fusion chapter:

- `05_sensor_fusion.md`

The purpose of sensor fusion in this project is to make planning stable even when camera detections are noisy, delayed, or temporarily missing.

Implementation reference:

- `robotpu-fusion.ts`
- `robotpu-kalman-filter.ts`
- `robotpu-soccer-mvp.ts`

Use the following fusion pattern:

1. Predict the current ball and goal state using the previous estimate.
2. Read the latest camera packet.
3. Reject invalid packets and treat stale packets with lower trust.
4. Use odometry to compensate latency between camera measurement time and the current time.

5. Smooth ball and goal positions with a lightweight filter.
6. Pass the filtered ball and goal estimates to the mapping and planning modules.

For micro:bit, the recommended filter is intentionally small:

- low-pass smoothing for the first working version,
- then optional 1D Kalman filters per axis for ball and goal position.

7.2.7 Full Navigation (MVP)

Read the MVP soccer chapter:

- 06_soccer_mvp.md

The full navigation program combines all previous modules into one control loop:

1. Read odometry from `robotPu.locationArray()`.
2. Read ball and goal detections from the ESP32 camera over I2C.
3. Gate packets using `VALID` and `STALE`.
4. Latency-compensate detections into the current local frame.
5. Smooth ball and goal estimates.
6. Build/update the local grid map.
7. Compute the kick point.
8. Plan a path or viewpoint sequence to the kick point.
9. Walk toward the target using bounded speed and turn commands.
10. Align with the goal and call `robotPu.kick()` or push the ball forward.

Implementation reference:

- `robotpu-soccer-demo.ts`
- `robotpu-soccer-mvp.ts`
- `robot-navigation.ts`

Recommended integration order:

1. Run ball following alone.
2. Add odometry-based latency compensation.
3. Add local map visualization/debug output.
4. Add kick-point planning.
5. Add filtered ball/goal estimates.
6. Add the final kick behavior.

When debugging, test each module separately before enabling the full behavior. If the robot behaves unpredictably, first check coordinate-frame conventions, packet validity flags, and whether `walkSpeed/walkTurn` signs match the RobotPU API.

7.2.8 Behavior Tree Planner

Read the behavior tree chapter:

- `07_soccer_bt.md`

Once the MVP flat planner is working, replace the `if/else` decision block with a proper Behavior Tree. The BT wraps the same actions and conditions into **Sequence** and **Fallback** composites, making the control flow explicit and easy to extend (e.g. fall recovery, opponent avoidance, goalie mode).

Key concepts:

- `btSequence([...])` — all children must succeed; stops on first FAILURE or RUNNING
- `btFallback([...])` — first succeeding child wins; stops on first SUCCESS or RUNNING
- Each condition node returns SUCCESS/FAILURE instantly (no side effects)
- Each action node sets `walkSpeed`, `walkTurn`, `walkMode` and returns RUNNING until done

Integration: replace the planner block in `robotpu-soccer-mvp.ts` with a single call to `btRoot()` each cycle. The actuator loop (`basic.forever`) remains unchanged.

Implementation reference:

- `07_soccer_bt.md` (full TypeScript listing)

8. References

- RobotPU JavaScript Documentation
- RobotPU Python Documentation
- Kalman Filter Tutorial

Chapter 2: Robot PU Odometry

Robot PU is a biped robot with two legs, whose yaw axes are 50 millimeters apart. It walks by alternating leg yaw movements with the help of feet roll movements. The odometry is calculated based on the leg yaw movements and yaw axis distance. When each step is completed, the robot's position is updated in the odometry frame.

For the final project, the minimum you need is:

1. Read pose from `robotPu.locationArray()`.
2. Understand that camera measurements are delayed.
3. Use odometry delta to compensate stale detections.

4. Be careful with heading conventions when mixing RobotPU pose readings with standard math formulas.

For example, the robot starts at position (0, 0) and facing direction 0 degrees. After one step, the robot yaws about the left leg rotation center (-25, 0) by 30 degrees, then the robot's position is updated to $(25\cos(30deg)-25, 25\sin(30deg))$ and facing direction 30 degrees. This is a simplified model; the actual odometry calculation is more complex.

Let's use transformation matrix to calculate the odometry. Basically keep updating transformation matrix from last pose to current pose. The left leg rotation center is at (-25, 0) in the robot frame, and the right leg rotation center is at (25, 0) in the robot frame. Then, the transformation matrix from last pose to current pose can be calculated as:

```
import numpy as np

def rot2(theta_rad: float) -> np.ndarray:
    c = np.cos(theta_rad)
    s = np.sin(theta_rad)
    return np.array([[c, -s], [s, c]], dtype=float)

def se2(R: np.ndarray, t: np.ndarray) -> np.ndarray:
    T = np.eye(3, dtype=float)
    T[0:2, 0:2] = R
    T[0:2, 2] = t
    return T

def trans2(tx: float, ty: float) -> np.ndarray:
    return se2(np.eye(2, dtype=float), np.array([tx, ty], dtype=float))

def rotate_about_pivot(delta_yaw_rad: float, pivot_xy_mm: np.ndarray) -> np.ndarray:
    """Step transform: Trans(p) * Rot(dtheta) * Trans(-p). Units: mm, rad."""
    px, py = float(pivot_xy_mm[0]), float(pivot_xy_mm[1])
    return trans2(px, py) @ se2(rot2(delta_yaw_rad), np.zeros(2)) @ trans2(-px, -py)

def left_leg_transformation_matrix(delta_yaw_rad: float, axis_half_distance_mm: float = 25.0) -> np.ndarray:
    pivot = np.array([-axis_half_distance_mm, 0.0], dtype=float)
    return rotate_about_pivot(delta_yaw_rad, pivot)

def right_leg_transformation_matrix(delta_yaw_rad: float, axis_half_distance_mm: float = 25.0) -> np.ndarray:
    pivot = np.array([axis_half_distance_mm, 0.0], dtype=float)
```

```

    return rotate_about_pivot(delta_yaw_rad, pivot)

def update_odometry(T_world_robot: np.ndarray, step_transformation_matrix: np.ndarray) -> np.ndarray:
    """Accumulate odometry:  $T_{k+1} = T_k @ T_{step}$ ."""
    return T_world_robot @ step_transformation_matrix

```

Study case 1: one step

Now assume the robot starts at position (0, 0) and facing direction 0 degrees. it used yaw angle of 30 degrees for left/right legs. move forward for 1 steps, where is the robot location and facing direction?

Assumption: the first step uses the left leg as the support pivot at (-25, 0) mm.

Verified using the Python code above:

```

• Location: (x, y) ~ (-3.349, 12.500) mm
• Heading: theta ~ 30 deg

T = np.eye(3, dtype=float)
T_step = left_leg_transformation_matrix(np.deg2rad(30.0), axis_half_distance_mm=25.0)
T = update_odometry(T, T_step)

x_mm, y_mm = float(T[0, 2]), float(T[1, 2])
theta_deg = float(np.rad2deg(np.arctan2(T[1, 0], T[0, 0])))

print("x,y (mm)=", x_mm, y_mm)
print("theta (deg)=", theta_deg)

```

Study case 2: two steps

Now assume the robot starts at position (0, 0) and facing direction 0 degrees. It uses yaw angle of +45 degrees for the left leg at (-25, 0) as the first step, then -45 degrees for the right leg at (25, 0) as the second step. Where is the robot location and facing direction?

In this toy model, each step rotates about a support pivot. So this study case is a two-step composition:

1. Step 1 (left support pivot at (-25, 0) mm, $\Delta\theta = +45^\circ$)
2. Step 2 (right support pivot at (25, 0) mm, $\Delta\theta = -45^\circ$)

Key observation:

- The net heading is $+45^\circ + (-45^\circ) = 0^\circ$ because the rotation parts multiply to identity: $R(+45^\circ)R(-45^\circ) = I$.

Verified result (using the code snippet below):

- After step 1: (x, y, theta) ~ (-7.322, 17.678, 45 deg) (mm, mm, deg)

- After step 2: (x, y, theta) ~ (-14.645, 35.355, 0 deg) (mm, mm, deg)

Verified using the Python code above:

```
T0 = np.eye(3, dtype=float)

# Step 1: left-support, +45 deg
T_step_L = left_leg_transformation_matrix(np.deg2rad(45.0), axis_half_distance_mm=25.0)
T_L = update_odometry(T0, T_step_L)

# Step 2: right-support, -45 deg
T_step_R = right_leg_transformation_matrix(np.deg2rad(-45.0), axis_half_distance_mm=25.0)
T_R = update_odometry(T_L, T_step_R)

def pose_from_T(T: np.ndarray):
    x_mm, y_mm = float(T[0, 2]), float(T[1, 2])
    theta_deg = float(np.rad2deg(np.arctan2(T[1, 0], T[0, 0])))
    return x_mm, y_mm, theta_deg

print("left-support:", pose_from_T(T_L))
print("right-support:", pose_from_T(T_R))
```

Study case 3: five steps

Now assume the robot starts at position (0, 0) and facing direction 0 degrees. it used yaw angle of 45 degrees for left legs at (-25,0) for 1st step, then move forward for 4 more steps with -90 degrees yaw for right leg at (25,0), 90 degrees yaw for left leg at (-25,0), with order of (right, left, right, left, ...), where is the robot location and facing direction?

Interpretation for this toy model:

- Step 1: left support, +45°
- Steps 2..5: each step, alternating support legs (right -90°, left 90°, right -90°, left 90°)

Verified using the Python code above:

```
• Location: (x, y) ~ (-7.322, 159.099) mm
• Heading: theta ~ 45 deg

T0 = np.eye(3, dtype=float)

T1 = update_odometry(T0, left_leg_transformation_matrix(np.deg2rad(45.0), axis_half_distance_mm=25.0))
T2 = update_odometry(T1, right_leg_transformation_matrix(np.deg2rad(-90.0), axis_half_distance_mm=25.0))
T3 = update_odometry(T2, left_leg_transformation_matrix(np.deg2rad(90.0), axis_half_distance_mm=25.0))
T4 = update_odometry(T3, right_leg_transformation_matrix(np.deg2rad(-90.0), axis_half_distance_mm=25.0))
T5 = update_odometry(T4, left_leg_transformation_matrix(np.deg2rad(90.0), axis_half_distance_mm=25.0))
```

```

x_mm, y_mm = float(T5[0, 2]), float(T5[1, 2])
theta_deg = float(np.rad2deg(np.arctan2(T5[1, 0], T5[0, 0])))

print("x,y (mm)=", x_mm, y_mm)
print("theta (deg)=", theta_deg)

```

Study case 4: turn

Now assume the robot starts at position (0, 0) and facing direction 0 degrees. it used yaw angle of 50 degrees for left legs at (-25,0) for 1st step, then move forward for 4 more steps with -80 degrees yaw for right leg at (25,0), 100 degrees yaw for left leg at (-25,0), with order of (right -80, left 100, right -80, left 100, ...), where is the robot location and facing direction?

Interpretation for this toy model:

- Step 1: left support, +50°
- Steps 2..5: right -80°, left +100°, right -80°, left +100°

Verified using the Python code above:

- Location: (x, y) ~ (-68.301, 143.969) mm
- Heading: theta ~ 90 deg

```
T0 = np.eye(3, dtype=float)
```

```

T1 = update_odometry(T0, left_leg_transformation_matrix(np.deg2rad(50.0), axis_half_distance))
T2 = update_odometry(T1, right_leg_transformation_matrix(np.deg2rad(-80.0), axis_half_distance))
T3 = update_odometry(T2, left_leg_transformation_matrix(np.deg2rad(100.0), axis_half_distance))
T4 = update_odometry(T3, right_leg_transformation_matrix(np.deg2rad(-80.0), axis_half_distance))
T5 = update_odometry(T4, left_leg_transformation_matrix(np.deg2rad(100.0), axis_half_distance))

```

```

x_mm, y_mm = float(T5[0, 2]), float(T5[1, 2])
theta_deg = float(np.rad2deg(np.arctan2(T5[1, 0], T5[0, 0])))

print("x,y (mm)=", x_mm, y_mm)
print("theta (deg)=", theta_deg)

```

We verified that the workflow works correctly by running the Python code upon common use cases. Now, let's consolidate the code into a class for better organization and reusability. We can create a class `RobotPUOdometry` that encapsulates the odometry calculation logic. The class should have the following methods:

- `__init__(self, axis_half_distance_mm: float = 25.0)` - Initialize the odometry with the axis half distance.
- `update(self, transformation_matrix: np.ndarray)` - Update the odometry with a transformation matrix. Can be used to handle odometry drift using external sensor data (camera, lidar, etc.).

- `left_step(self, yaw_angle_deg: float)` - Perform a left leg step with the given yaw angle.
- `right_step(self, yaw_angle_deg: float)` - Perform a right leg step with the given yaw angle.
- `get_position(self)` -> `tuple[float, float, float]` - Get the current position (x, y, theta) in millimeters and degrees from current transformation matrix.
- `reset(self)` - Reset the odometry to the initial state.

```
class RobotPUOdometry:
    def __init__(self, axis_half_distance_mm: float = 25.0):
        self.axis_half_distance_mm = axis_half_distance_mm
        self.current_transformation = np.eye(3, dtype=float)

    def update(self, transformation_matrix: np.ndarray):
        self.current_transformation = update_odometry(self.current_transformation, transformation_matrix)

    def left_step(self, yaw_angle_deg: float):
        self.current_transformation = update_odometry(self.current_transformation, left_leg_transformation(yaw_angle_deg))

    def right_step(self, yaw_angle_deg: float):
        self.current_transformation = update_odometry(self.current_transformation, right_leg_transformation(yaw_angle_deg))

    def get_position(self) -> tuple[float, float, float]:
        x_mm, y_mm = float(self.current_transformation[0, 2]), float(self.current_transformation[1, 2])
        theta_deg = float(np.rad2deg(np.arctan2(self.current_transformation[1, 0], self.current_transformation[0, 0])))
        return x_mm, y_mm, theta_deg

    def reset(self):
        self.current_transformation = np.eye(3, dtype=float)
```

How to use the class:

- Initialize the class with the axis half distance: `odometry = RobotPUOdometry(axis_half_distance_mm=25.0)`
- Reset the odometry: `odometry.reset()`
- Perform steps: `odometry.left_step(yaw_angle_deg=50.0)` or `odometry.right_step(yaw_angle_deg=-80.0)`
- Update the odometry with a defined external correction transform if you have one from another sensor.
- Get the current position: `odometry.get_position()`

For example, to solve study case 2:

```
odometry = RobotPUOdometry(axis_half_distance_mm=25.0)
odometry.reset()
odometry.left_step(yaw_angle_deg=45.0)
odometry.right_step(yaw_angle_deg=-45.0)
x_mm, y_mm, theta_deg = odometry.get_position()
```


Now let's use this code to solve study case 4:

```
odometry = RobotPUOdometry(axis_half_distance_mm=25.0)
odometry.reset()
odometry.left_step(yaw_angle_deg=50.0)
odometry.right_step(yaw_angle_deg=-80.0)
odometry.left_step(yaw_angle_deg=100.0)
odometry.right_step(yaw_angle_deg=-80.0)
odometry.left_step(yaw_angle_deg=100.0)
x_mm, y_mm, theta_deg = odometry.get_position()
```

Compare the output with the expected result from study case 4 to verify the implementation is correct.

Once the workflow validated in python, we can port it to javascript for micro:bit. The javascript implementation should be very similar to the python implementation, with the same API and logic, just no numpy dependency, and using javascript's built-in math functions to add matrix operations instead of numpy.

The javascript implementation for micro:bit is :

JavaScript implementation notes (micro:bit / MakeCode)

Python + numpy hides a lot of details (matrix creation, multiplication, slicing). On micro:bit we write those operations explicitly with plain arrays, but the math is the same.

1) What is stored in currentTransformation `currentTransformation` is the SE(2) pose matrix `T_world_robot`:

$$T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$$

- `R` is a 2x2 rotation matrix (`rot2`)
- `t` = `[x_mm, y_mm]` is translation in millimeters

So extracting pose is exactly:

- `x_mm` = `T[0][2]`
- `y_mm` = `T[1][2]`
- `theta_rad` = `atan2(R[1][0], R[0][0])`

This is what `getPosition()` does.

1.1) Angle convention note (standard convention + RobotPU convention) The SE(2) matrix math uses the standard 2D convention:

- `theta_std` = 0 points along +X
- positive `theta_std` is CCW toward +Y
- bearing to a point is `atan2(dy, dx)`

RobotPU firmware reports `theta_robot_deg` with 0 meaning +Y (forward). When mixing RobotPU pose readings with standard math formulas, convert with:

- `theta_std_deg = 90 - theta_robot_deg`
- `theta_std_rad = deg2rad(theta_std_deg)`

2) Why helpers are static methods (avoids “function re-define”) In MakeCode, all `.ts` files are compiled into one global namespace.

- If you write `function deg2rad(...)` in multiple files, you can easily hit “function already defined” / “duplicate implementation” errors.
- Putting helpers under `Odometry.*` keeps the global namespace clean and makes it clear which file owns the math.

3) How a step is modeled: rotate about a support-foot pivot Each step is simplified as a pure rotation of the robot body about a fixed pivot point (the support foot yaw axis) in the **robot frame**:

`T_step = Trans(pivot) · Rot(deltaYaw) · Trans(-pivot)`

This is implemented by `rotateAboutPivot(deltaYawRad, pivotXYmm)`.

Then odometry accumulation is the standard SE(2) composition:

`T_{k+1} = T_k · T_step`

implemented in `updateOdometry(...)` and used by `update(...)`.

4) How to use on micro:bit The intended usage pattern is the same as Python:

```
let odom = new Odometry(25)
odom.reset()
```

```
// Call one of these each time your gait completes a left/right support step.
odom.leftStep(deltaYawDeg)
odom.rightStep(deltaYawDeg)
```

```
let p = odom.getPosition()
// p.x_mm, p.y_mm, p.theta_deg
```

Important: `leftStep(...)` / `rightStep(...)` assume the parameter is the **yaw change (delta)** for that step (in degrees). If your gait code works in raw servo angles, convert them to a delta yaw first.

```
// RobotPU odometry reference implementation (micro:bit-friendly JavaScript).
//
// This file is a direct port of the Python/Numpy version (SE(2) homogeneous transforms),
// but implemented using plain JavaScript arrays (no numpy dependency).
```

```

//
// NOTE: This file is compiled in a MakeCode-like environment where all .ts files share
// one global namespace. To avoid "function re-define" problems, helpers are kept as
// static methods on Odometry (instead of top-level functions).

class Odometry {
  public axisHalfDistanceMm: number;
  public currentTransformation: number[] [];

  static rot2(thetaRad: number): number[] [] {
    let c = Math.cos(thetaRad);
    let s = Math.sin(thetaRad);
    return [
      [c, -s],
      [s, c],
    ];
  }

  static se2(R: number[] [], t: number[]): number[] [] {
    return [
      [R[0][0], R[0][1], t[0]],
      [R[1][0], R[1][1], t[1]],
      [0, 0, 1],
    ];
  }

  static trans2(tx: number, ty: number): number[] [] {
    return [
      [1, 0, tx],
      [0, 1, ty],
      [0, 0, 1],
    ];
  }

  static matMul3(A: number[] [], B: number[] []): number[] [] {
    return [
      [
        A[0][0] * B[0][0] + A[0][1] * B[1][0] + A[0][2] * B[2][0],
        A[0][0] * B[0][1] + A[0][1] * B[1][1] + A[0][2] * B[2][1],
        A[0][0] * B[0][2] + A[0][1] * B[1][2] + A[0][2] * B[2][2],
      ],
      [
        A[1][0] * B[0][0] + A[1][1] * B[1][0] + A[1][2] * B[2][0],
        A[1][0] * B[0][1] + A[1][1] * B[1][1] + A[1][2] * B[2][1],
        A[1][0] * B[0][2] + A[1][1] * B[1][2] + A[1][2] * B[2][2],
      ],
    ];
  }
}

```

```

        [
            A[2][0] * B[0][0] + A[2][1] * B[1][0] + A[2][2] * B[2][0],
            A[2][0] * B[0][1] + A[2][1] * B[1][1] + A[2][2] * B[2][1],
            A[2][0] * B[0][2] + A[2][1] * B[1][2] + A[2][2] * B[2][2],
        ],
    ];
}

static rotateAboutPivot(deltaYawRad: number, pivotXYmm: number[]): number[][] {
    let px = pivotXYmm[0];
    let py = pivotXYmm[1];
    return Odometry.matMul3(
        Odometry.matMul3(
            Odometry.trans2(px, py),
            Odometry.se2(Odometry.rot2(deltaYawRad), [0, 0])
        ),
        Odometry.trans2(-px, -py)
    );
}

static updateOdometry(TworldRobot: number[][], stepTransformationMatrix: number[][]): number[][] {
    return Odometry.matMul3(TworldRobot, stepTransformationMatrix);
}

static deg2rad(deg: number): number {
    return (deg * Math.PI) / 180.0;
}

static rad2deg(rad: number): number {
    return (rad * 180.0) / Math.PI;
}

static identity3(): number[][] {
    return [
        [1, 0, 0],
        [0, 1, 0],
        [0, 0, 1],
    ];
}

constructor(axisHalfDistanceMm: number = 25.0) {
    this.axisHalfDistanceMm = axisHalfDistanceMm;
    this.currentTransformation = Odometry.identity3();
}

update(transformationMatrix: number[][]): void {

```

```

        // Apply a general SE(2) step transform (e.g., external correction).
        this.currentTransformation = Odometry.updateOdometry(this.currentTransformation, tra
    }

    leftStep(yawAngleDeg: number): void {
        // Apply one walking step where the left leg is the support pivot.
        this.update(Odometry.rotateAboutPivot(Odometry.deg2rad(yawAngleDeg), [-this.axisHalf
    }

    rightStep(yawAngleDeg: number): void {
        // Apply one walking step where the right leg is the support pivot.
        this.update(Odometry.rotateAboutPivot(Odometry.deg2rad(yawAngleDeg), [this.axisHalf
    }

    getPosition(): { x_mm: number; y_mm: number; theta_deg: number } {
        // Return (x,y,theta) extracted from the SE(2) matrix.
        let xMm = this.currentTransformation[0][2];
        let yMm = this.currentTransformation[1][2];
        let thetaDeg = Odometry.rad2deg(Math.atan2(this.currentTransformation[1][0], this.cu
        return { x_mm: xMm, y_mm: yMm, theta_deg: thetaDeg };
    }

    reset(): void {
        this.currentTransformation = Odometry.identity3();
    }
}

```

Chapter 3: Local Mapping and Latency Compensation on Robot PU

Robot PU is equipped with a **BBC micro:bit (V2)** serving as the central execution and navigation processor, an **ESP32-S3 Eye / Camera** for vision processing, and a **Sonar (Ultrasonic) sensor** for obstacle avoidance. In this chapter, we focus on the **local mapping** pipeline used by the current MVP implementation (camera packet -> ground-plane 2D -> 10x10 grid), plus **odometry-based latency compensation**. We intentionally keep **sensor fusion** and **navigation/obstacle-handling** out of this chapter:

- Path planning and following is covered in `04_path.md`.
- Sensor fusion (odometry + camera + sonar robustness) is covered in `05_sensor_fusion.md`.

This separation matters because the ball can look like an “obstacle” in a naive occupancy grid, but in navigation you sometimes must intentionally bump the ball.

Quick overview (what you are building)

This is an ambitious but practical system because you offload heavy vision work to the ESP32-S3 and keep the micro:bit focused on a small grid map and reactive navigation.

At a high level, your controller loop will:

1. Read an ESP32 detection packet over I2C.
2. Interpret (`x_mm`, `y_mm`) as a ground-plane 2D point in the camera/robot-local frame.
3. Use odometry to compensate camera latency and express the detection in the current robot-local frame.
4. Map ball/goal/kick-point into a small discrete grid.
5. Navigate to the kick point, align to the goal, then kick.

Feasibility notes (micro:bit V2 limits)

The micro:bit V2 has tight RAM constraints (on the order of 65 KB usable for your program and runtime). To make this project feasible:

- Keep data structures **small and static**:
 - 10 × 10 grids are fine.
 - Avoid large arrays, images, strings, or storing long histories.
- Avoid allocating new objects inside fast loops:
 - Prefer reusing buffers and variables inside `basic.forever`.
- Treat this lab as **local mapping + reactive navigation**, not full SLAM:
 - The ESP32 does vision.
 - The micro:bit fuses a few measurements, maintains a tiny grid, and issues motion commands.

Minimal viable pipeline (dependency order)

Implement the system in this order so each step provides the inputs for the next:

1. **Read detections from ESP32 over I2C**
 - Parse the packet, check `VALID` / `STALE`.
2. **Project to a 2D ground plane (MVP simplification)**
 - Use (`x_mm`, `y_mm`) from the packet as a 2D point.
 - Convention used in the reference code: `x_mm` = left/right, `y_mm` = forward.
3. **Latency compensation using odometry**
 - Store the odometry pose when the packet arrives.
 - Use odometry delta to transform the stale local detection into the current local frame.
4. **Map into the 10 × 10 grid**
 - Mark ball/goal/kick-point; keep obstacles as a separate layer.
5. **Kick target computation**

- Compute a point behind the ball along the goal->ball direction.
6. **Navigate**
- Pass the mapped ball/goal/kick-point to the navigation module (see `04_path.md` and `06_soccer_demo.md`).

Code organization (recommended):

- Follow ball helpers (camera -> ball location): `robotpu-followball.ts`
- Odometry SE(2) implementation: `robotpu-odometry.ts`
- Mapping helpers (ball location -> grid): `robotpu-localmap.ts`

Game rules (10x10 grid setup)

This project uses a 10×10 grid as a simplified soccer field (1 m x 1 m, so each cell is about 10 cm x 10 cm). Grid coordinates are written as (x, y) with each index in $[0, 9]$.

- **Goal center location:** $(0, 5)$ with size of A4 paper (210 mm height x 297 mm width).
- **Ball placement:** the soccer ball is placed at a random mid-field cell (randomized by the instructor/your test harness).
- **Robot placement:** Robot PU is placed at a random cell on the 10th row, i.e. $(9, y)$ where $y \in [0, 9]$.
- **Initial orientation / visibility:** the robot starts facing such that the soccer ball and the goal are both guaranteed to be in the camera view at the start.
- **Objective:** navigate to a good kick position, then push the ball into the goal.

1. System Architecture & Physical Configurations

To build an intelligent autonomous robot, we must align all sensors and actuators into a single geometric understanding of the environment.

1.1 Fixed Physical Offsets

The ESP32-S3 Camera is rigidly mounted to the robot's frame. In a full model you should account for its offset and head yaw/pitch.

However, the current MVP code (`robotpu-localmap.ts`) assumes the robot tries to keep its head facing forward while walking, and uses the **packet-provided** (`x_mm`, `y_mm`) directly as a local 2D ground-plane measurement.

- $X_{\text{offset}} = 0.000$ meters (Centered)
- $Y_{\text{offset}} = 0.035$ meters (3.5 cm forward from center)
- $Z_{\text{offset}} = 0.160$ meters (16.0 cm above the ground)

1.2 Object Dimensions & Detection Targets

- **Soccer Ball:** Standard spherical radius or diameter. Diameter = 0.05 m (5 cm).
- **Soccer Goal:** Width = 0.297 m (30 cm), Height = 0.210 m (20 cm).

1.3 I2C Camera API

The I2C camera API is defined in `robotpu-i2c-cam.ts`. Conceptually:

- The **ESP32-S3** runs the vision model and acts as an **I2C slave**.
- The **micro:bit** polls the ESP32 by reading a fixed-size packet from `ESP32_ADDR`.
- Each packet encodes a single “event” (e.g., soccer ball detection or soccer goal detection).

1.3.1 Addresses and polling

Hardware addresses used in the reference code:

- MUX (TCA9546A): `MUX_ADDR = 112 (0x70)`
- ESP32 camera module: `ESP32_ADDR = 66 (0x42)`

Before polling, the code opens the I2C mux channels:

```
pins.i2cWriteNumber(MUX_ADDR, 0x0F, NumberFormat.Int8LE, false)
```

Then you poll in a loop (about every 20 ms in the reference):

```
let packet = pins.i2cReadBuffer(ESP32_ADDR, SIZE, false)
if (packet.length == SIZE) {
  // parse fields...
}
basic.pause(20)
```

1.3.2 Enabling services (turning on detections)

The ESP32 starts with services disabled after reboot (by default **all detections are OFF**). Enable only what you need by writing a short command packet:

```
pins.i2cWriteBuffer(
  ESP32_ADDR,
  Buffer.fromArray([CMD_SERVICE_ENABLE, serviceId, enabled ? 1 : 0]),
  false
)
```

In `robotpu-i2c-cam.ts`, the demo enables:

- image capture
- soccer ball detection
- soccer goal detection

For the soccer lab, a good default is:

- turn **ON** `SERVICE_SOCCER_BALL_DETECTION`

- turn **ON** SERVICE_SOCCER_GOAL_DETECTION
- turn **OFF** SERVICE_FACE_DETECTION (save compute/bandwidth and avoid irrelevant packets)

1.3.3 Packet types (events) The packet begins with a **type** byte:

- SOCCER_BALL = 0x04
- SOCCER_GOAL = 0x05

Other event types exist (face, wake, voice) but the soccer MVP mostly uses the two above.

1.3.4 Packet validity and staleness (must understand) Each packet has a **flags** byte that tells you if the data should be trusted:

- **VALID** (bit 0): the payload contains a real detection
- **STALE** (bit 1): the payload is an older repeated result (useful for UI, but risky to treat as a new measurement)

Recommended student policy:

- If `(flags & VALID) == 0`: treat as “no measurement” this cycle.
- If `(flags & STALE) != 0`: do not overwrite your state estimate with it.

This is exactly what the reference printer does:

```
if (!(flags & STALE)) {
    serial.writeLine("fresh detection")
}
```

Why it matters:

- Your control loop runs faster than vision.
- Without STALE gating, you may “re-apply” the same measurement repeatedly and cause oscillations.

1.3.5 Parsing ball/goal position from the packet When **type** is SOCCER_BALL or SOCCER_GOAL, the packet includes:

- **count**: number of detected objects in this frame
- **score**: confidence/quality score
- **(x_mm, y_mm, z_mm)**: estimated object position in millimeters
- **(w, h)**: bounding box size
- **(yaw, pitch)**: suggested head correction (bytes interpreted as signed int8)

Reference parsing code:

```
let count = p[4]
let score = p[5]
let x_mm = i16(p, 6)
```

```

let y_mm = i16(p, 8)
let z_mm = i16(p, 10)
let w = u16(p, 12)
let h = u16(p, 14)
let yaw = i8(p[16])
let pitch = i8(p[17])

```

For local mapping (MVP), the most important fields are `x_mm` and `y_mm`:

- `x_mm` is left/right
- `y_mm` is forward distance

Convert to meters:

```

let x_m = x_mm / 1000
let y_m = y_mm / 1000

```

Then treat `(x_m, y_m)` as a 2D ground-plane measurement in the robot/camera-local frame.

2. Mathematical Foundation

2.1 Coordinate transformations (optional full model)

The ESP32-S3 Camera can return object coordinates in its own **camera frame** (X_c, Y_c, Z_c) . If you want a more complete geometric model, you can use head **pitch** (θ) and **yaw** (ϕ) and camera mounting offset to transform into a robot base frame.

Given a point detected by the camera (X_c, Y_c, Z_c) , we first apply the rotation transformations based on the head angles, and then add the fixed structural translation offsets.

Only use the following formulas if your camera-frame axes and yaw/pitch signs match these assumptions. For the MVP, use the packet-provided `(x_mm, y_mm)` directly as described below:

$$X_r = (X_c \cdot \cos \phi - Y_c \cdot \sin \phi) + X_{\text{offset}}$$

$$Y_r = (X_c \cdot \sin \phi \cdot \sin \theta + Y_c \cdot \cos \phi \cdot \sin \theta + Z_c \cdot \cos \theta) + Y_{\text{offset}}$$

$$Z_r = (X_c \cdot \sin \phi \cdot \cos \theta + Y_c \cdot \cos \phi \cdot \cos \theta - Z_c \cdot \sin \theta) + Z_{\text{offset}}$$

For our simplified 2D ground navigation plane, you can use the extracted X_r and Y_r positions.

MVP simplification used by `robotpu-localmap.ts`:

- Skip the 3D transform and treat the packet fields (`x_mm`, `y_mm`) as a local 2D measurement.
- This works best when head yaw/pitch stays near zero during walking.

Micro:bit implementation note:

- A reference implementation of these mapping utilities (camera transform, projection, `worldToGrid`, kick target) is in `robotpu-map.ts`.

2.1.1 Practical recipe (MVP): local 2D from ESP32 packet The reference programs `robotpu-followball.ts` and `robotpu-localmap.ts` show how to read:

- `x_mm`, `y_mm` (ball/goal position estimate in the camera-local frame, units: mm)
- packet flags `VALID`/`STALE`

MVP policy:

1. If the packet is not `VALID`, treat it as “no measurement”.
2. Convert to meters:
 - $x_m = x_{mm} / 1000$, $y_m = y_{mm} / 1000$
3. Treat (`x_m`, `y_m`) as a local ground-plane point (left/right, forward).

Reliability rule (important for feasibility):

- If the packet is not `VALID`, do not update state.
- If `STALE`, do not overwrite state aggressively (keep last good value or down-weight).

2.2 Odometry-based latency compensation (what the MVP code actually uses)

Camera results can be delayed (about 0.3 s). The MVP code compensates this by storing:

- the local measurement (ball/goal) at the moment the packet is received, and
- the robot odometry pose at that same moment.

Then in the control loop, it transforms that stale measurement into the current local frame.

This is implemented in `robotpu-localmap.ts` as:

- storing `ball_cam2D` / `goal_cam2D` (meters)
- storing `ball_pose_0` / `goal_pose_0` (odometry pose at receipt)
- transforming via `camToNow(cam2D_at_det, pose_det_0, pose_now_0)`

Conceptually:

- This gives $(\mathbf{x}, \mathbf{y})$ in the current local frame, which is then used for grid indexing and control.

To successfully push the ball directly into the goal center, Robot PU must not simply charge at the ball. It must navigate to a calculated **Striking Target Position** exactly 2 inches (≈ 0.05 meters) directly behind the ball, aligned perfectly with the vector facing the goal center.

1. Let the Ball position be $\vec{B} = (X_b, Y_b)$ and Goal center position be $\vec{G} = (X_g, Y_g)$.
2. Compute the directional vector from the Goal to the Ball:

$$\Delta Y = Y_b - Y_a$$

- $$D = \sqrt{(\Delta X)^2 + (\Delta Y)^2}$$

- $$X_{\text{kick}} = X_b + \left(\frac{\Delta X}{D} \right) \cdot 0.05$$

$$Y_{\text{kick}} = Y_b + \left(\frac{\Delta Y}{D} \right) \cdot 0.05$$

Your 10×10 grid is an **occupancy map**: each cell stores a belief about whether the space is free or occupied. This is exactly the Class 11 occupancy-grid idea, just at a much lower resolution.

2.4.1 Forward model vs inverse sensor model Class 11 distinction:

- **Forward sensor model:** $p(z_t \mid m, x_t)$, predicts what the sensor would measure if the map m were known.
- **Inverse sensor model:** $p(m_i \mid z_t, x_t)$, directly converts a measurement into an update for each cell m_i .

For range sensors (sonar/LiDAR), a standard inverse sensor model intuition is:

- Cells **along the ray before the hit** get **free-space evidence**.
- The **endpoint cell** gets **occupied evidence**.
- Untouched cells get no update.

For the soccer lab, you can apply the same idea to:

- **Sonar:** gives obstacle evidence along the robot heading.
- **Vision (ball/goal):** gives **object location evidence** in a cell (not a full ray model unless you want to model the camera frustum).

2.4.2 Log-odds representation (why it helps) In Class 11, we store occupancy belief in **log-odds** to make repeated updates additive:

$$l = \log \left(\frac{p}{1-p} \right)$$

Conversion back to probability uses the logistic:

$$p = \frac{1}{1 + \exp(-l)}$$

Log-odds update rule (Class 11 form):

$$l_t(i) = l_{t-1}(i) + l(z_t, x_t, i) - l_0$$

where l_0 is the prior (typically $l_0 = 0$ meaning $p_0 = 0.5$, “unknown”).

Practical takeaway for micro:bit:

- Use a **small integer** or **fixed-point** log-odds grid (e.g., -20..+20) to avoid floats.
- Increment by constants on each observation: $L \mathrel{+}= L_OCC$ for occupied, $L \mathrel{+}= L_FREE$ for free.

2.4.3 How to fuse sonar + camera without heavy SLAM You are not doing full SLAM here; you are doing **local mapping** around the robot.

- **Sonar** is great for marking **obstacles** (occupied evidence).
- **Camera** is great for marking **targets** (ball/goal positions) and for closed-loop tracking.

Because you refresh the grid frequently, a simple approach works well:

- Keep an occupancy belief grid for obstacles.
- Keep the ball/goal as **separate state** (or a separate grid layer) so it does not get “stuck” as an obstacle.

2.5 Getting ball/goal detections from ESP32-S3 over I2C (how the MVP code does it)

The reference implementations are:

- `robotpu-followball.ts` (head-lock + follow-ball)
- `robotpu-localmap.ts` (local mapping + latency compensation + kick-point planning)

Conceptually, the ESP32-S3 acts as an **I2C slave** that streams a detection packet. The micro:bit:

1. Reads a fixed-size I2C packet.
2. Checks if the packet is valid/stale.
3. Extracts an estimated ball/goal position (in mm) and optional steering offsets.
4. Optionally uses the packet-provided yaw/pitch offsets to move head yaw/pitch to keep the ball centered.

In the code, the detection packet fields are parsed like this:

```
let x_mm = i16(p, 6)
let y_mm = i16(p, 8)
let z_mm = i16(p, 10)
let w = u16(p, 12)
let h = u16(p, 14)
let yaw = i8(p[16])
let pitch = i8(p[17])
```

How to interpret them (typical use):

- `x_mm`, `y_mm`, `z_mm`: an estimated 3D location of the object in the camera frame (units: mm).
- `w`, `h`: bounding box size (useful as a sanity check or confidence cue).
- `yaw`, `pitch`: suggested image-plane correction that the code uses to servo the head.

MVP simplification used in `robotpu-localmap.ts`:

- treat `(x_mm, y_mm)` as a 2D ground-plane measurement in the robot/camera-local frame
- convert to meters: `(x_m, y_m) = (x_mm/1000, y_mm/1000)`

Important robustness idea (fits Class 11 “measurement validity” mindset): use the packet **flags**.

```
let flags = p[3]
const VALID = 1 << 0
const STALE = 1 << 1
```

Recommended policy:

- If **VALID** is not set: treat as “no measurement” (do not update ball state).
- If **STALE** is set: down-weight the measurement (e.g., slow head motion, keep previous ball estimate).

This is the same principle as Class 11: **not all measurements are equally trustworthy**, so don’t overwrite your belief/state aggressively.

3. Implementation roadmap (MVP first)

Do not try to implement everything at once. Follow this order so that each part has the information it depends on.

3.1 MVP checklist (what must work first)

1. **I2C read + packet validity**
 - Read the ESP32 packet.
 - If not **VALID**, treat as “no measurement”.
2. **2D measurement in the local frame (MVP)**
 - Parse `x_mm`, `y_mm` and convert to meters.
 - Treat it as a local ground-plane point (`x` left/right, `y` forward).
3. **Latency compensation using odometry (recommended)**
 - When a packet arrives, store the odometry pose.
 - In the main loop, transform the stored local point into the current local frame (see `camToNow` in `robotpu-localmap.ts`).
4. **Grid mapping**
 - Map the current-frame ball/goal into the 10×10 grid.
5. **Kick target computation**
 - Compute “behind the ball” target from ball and goal.
6. **Navigation**
 - Implement as a separate module (see `04_path.md` and `06_soccer_demo.md`). Obstacle handling and robustness belongs in `05_sensor_fusion.md`.

3.2 Stretch goals (add only after MVP)

- **Log-odds obstacle layer** for sonar (Class 11 style belief updates instead of hard overwrites).
- **Ball/goal filtering** (temporal smoothing; down-weight **STALE** packets).
- **Search behavior** when the ball is lost (scan pattern).
- **Better planning** (e.g., A* on a tiny grid) if your obstacle layer is stable.

3.3 Full reference program (optional)

The real reference implementation for this chapter is `robotpu-localmap.ts`. It uses a deliberate design choice:

- **plan in the current camera-local frame** (a local frame tied to what the camera sees now),
- **do not depend on global localization**, and
- compensate camera latency using **odometry deltas**.

Below is a condensed, student-friendly version of the key pieces. You should implement these pieces incrementally.

A) Store detections at receipt time (camera frame) + odometry pose

The ESP32 packet arrives late, so we keep:

- `*_cam2D`: the ball/goal 2D position in the camera frame at detection time (meters)
- `*_pose_0`: the robot odometry pose when we received that packet

```
let ball_cam2D: number[] = [0, 0]
let ball_pose_0: number[] = [0, 0, 0]
let ball_valid = false
let ball_rx_ms = 0
```

```
let goal_cam2D: number[] = [0, 0]
let goal_pose_0: number[] = [0, 0, 0]
let goal_valid = false
let goal_rx_ms = 0
```

When a VALID and non-STALE packet arrives, save `(x_mm,y_mm)` converted to meters and also snapshot odometry:

```
const pose0 = getPose0() // [x,y,theta] in odometry frame
ball_cam2D = [x_mm / 1000, y_mm / 1000] // x: left/right, y: forward
ball_pose_0 = [pose0[0], pose0[1], pose0[2]]
ball_valid = true
ball_rx_ms = input.runningTime()
```

B) Latency compensation: transform the old local point into the current local frame This is the core mapping trick. Convert the stale local point into odometry, then into the current robot-local frame:

```
function rot(theta: number, x: number, y: number): number[] {
  const c = Math.cos(theta)
  const s = Math.sin(theta)
  return [c * x - s * y, s * x + c * y]
}
```



```

function camToNow(cam2D_at_det: number[], pose_det_0: number[], pose_now_0: number[]): number[] {
  // 1) local@det -> odom
  const det_xy_0 = rot(pose_det_0[2], cam2D_at_det[0], cam2D_at_det[1])
  const obj_0 = [pose_det_0[0] + det_xy_0[0], pose_det_0[1] + det_xy_0[1]]

  // 2) odom -> local@now
  const rel_0 = [obj_0[0] - pose_now_0[0], obj_0[1] - pose_now_0[1]]
  const rel_now = rot(-pose_now_0[2], rel_0[0], rel_0[1])
  return rel_now
}

```

Now you can compute `ball_now` and `goal_now` every loop:

```

const pose_now_0 = getPose0()
let ball_now = ball_valid ? camToNow(ball_cam2D, ball_pose_0, pose_now_0) : [0, 0]
let goal_now = goal_valid ? camToNow(goal_cam2D, goal_pose_0, pose_now_0) : [0, 0]

```

C) Build a tiny 10x10 debug grid in the *current* local frame The reference uses a small grid mainly for debugging/visualization (0 = empty, 2 = ball, 3 = goal, 4 = kick target).

```

const GRID_N = 10
const GRID_RES_M = 0.05
const GRID_HALF_M = (GRID_N * GRID_RES_M) / 2

function gridIndex(x_m: number, y_m: number): number[] {
  // x: left/right, y: forward
  const j = Math.floor((x_m + GRID_HALF_M) / GRID_RES_M)
  const i = Math.floor((y_m) / GRID_RES_M)
  return [i, j]
}

```

D) Compute the kick point (behind the ball along goal->ball)

```

const KICK_BACKOFF_M = 0.05
function norm2(x: number, y: number): number { return Math.sqrt(x * x + y * y) }

function computeKickPoint(ball_now: number[], goal_now: number[]): number[] {
  const dx = ball_now[0] - goal_now[0]
  const dy = ball_now[1] - goal_now[1]
  const n = Math.max(1e-6, norm2(dx, dy))
  const ux = dx / n
  const uy = dy / n
  return [ball_now[0] + ux * KICK_BACKOFF_M, ball_now[1] + uy * KICK_BACKOFF_M]
}

```

E) Minimal closed-loop behavior (MVP) This is not a full navigation stack yet. It is an MVP that uses the local frame:

- if ball+goal: go to kick point, then face goal, then robotPu.kick()
- if ball only: approach the ball
- otherwise: stop + scan

```
function desiredHeadingTo(x: number, y: number): number {
  // heading in robot-local frame (standard convention): atan2(dy, dx)
  return Math.atan2(y, x)
}

basic.forever(function () {
  const now_ms = input.runningTime()
  const pose_now_0 = getPose0()

  // Drop measurements that are too old.
  if (ball_valid && (now_ms - ball_rx_ms) > 6000) ball_valid = false
  if (goal_valid && (now_ms - goal_rx_ms) > 6000) goal_valid = false

  const ball_now = ball_valid ? camToNow(ball_cam2D, ball_pose_0, pose_now_0) : [0, 0]
  const goal_now = goal_valid ? camToNow(goal_cam2D, goal_pose_0, pose_now_0) : [0, 0]

  if (ball_valid && goal_valid) {
    const kickPt = computeKickPoint(ball_now, goal_now)
    const distKick = norm2(kickPt[0], kickPt[1])
    const headingKick = desiredHeadingTo(kickPt[0], kickPt[1])
    const turn = clamp(-1.2 * headingKick, -0.8, 0.8)

    if (distKick > 0.11) {
      const speed = distKick > 0.25 ? 2.5 : 1.5
      robotPu.walk(speed, turn)
    } else {
      const headingGoal = desiredHeadingTo(goal_now[0], goal_now[1])
      const turnGoal = clamp(-1.2 * headingGoal, -0.8, 0.8)
      if (Math.abs(headingGoal) > 0.25) robotPu.walk(1.0, turnGoal)
      else robotPu.kick()
    }
  } else if (ball_valid) {
    const distBall = norm2(ball_now[0], ball_now[1])
    const headingBall = desiredHeadingTo(ball_now[0], ball_now[1])
    const turnBall = clamp(-1.2 * headingBall, -0.8, 0.8)
    const speedBall = distBall > 0.25 ? 2.0 : 1.0
    robotPu.walk(speedBall, turnBall)
  } else {
    robotPu.walk(0, 0)
    searchBall(SEARCH_PATTERN)
  }
})
```

```

    }

    basic.pause(20)
  })

```

4. Calibration & Testing Protocol

To ensure your code functions properly on physical hardware, use the following validation roadmap:

1. **Verify axis conventions (x left/right, y forward):** Place the ball to the robot's left, then print `ball_cam2D[0]` (or `ball_now[0]`). It should be positive. Place the ball to the right; it should be negative.
2. **Validate distance scaling (mm to meters):** Place the ball about 0.5 m in front and check `ball_cam2D[1]` (or `ball_now[1]`) is near 0.5.
3. **Validate grid indexing:** Print `gridIndex(ball_now[0], ball_now[1])`. The `i` index should increase as the ball is further forward; the `j` index should increase as the ball moves left.
4. **Validate latency compensation:** While walking forward, hold the ball fixed in the world. Without `camToNow`, the ball appears to drift in your local map. With `camToNow`, `ball_now` should remain much more stable.
5. **Validate turning sign:** If the robot turns away from the target, flip the sign of your turn gain (in `robotpu-localmap.ts` this is the comment next to `TURN_GAIN`).

Chapter 4: Path Planning and Following

For the RobotPU soccer final project, the navigation goal is practical rather than time-optimal: move from the current robot pose to a useful viewpoint, such as the kick point behind the ball, while keeping walking commands bounded and stable.

Pure Pursuit and A* are still useful as lightweight planning tools, but they must be wrapped with a walking-safe controller that respects RobotPU's turning and speed limits.

For a walking robot, advanced time-optimal planning requires treating stepping capabilities as dynamic constraints.

For the MVP, we do not implement DCM or MPC on the micro:bit. We use a small grid planner, a viewpoint controller, and conservative command limits. DCM/MPC ideas are included later only as advanced background.

When viewpoints along the path are discretized, the path following problem becomes a sequence of footstep planning and execution problems. Each footstep must be planned to maintain balance and respect the robot's physical

constraints.

The practical MVP algorithms are described first. Advanced biped planning ideas are summarized at the end.

Follow Viewpoints

Move from Viewpoint A to Viewpoint B

The simplest navigation workflow we can run on micro:bit is to discretize a path into a list of viewpoints (poses) and then repeatedly solve the sub-problem:

Given:

- start pose: (`x`, `y`, `theta`) from `robotPu.locationArray()`
- goal pose: (`x`, `y`, `theta`) from the next viewpoint

Output:

- `walkSpeed`, `walkTurn` for `robotPu.walk(walkSpeed, walkTurn)`

In `robotpu-viewpoint.ts`, this is implemented as a pose-to-pose controller:

- It computes a direction to a “virtual target” near the goal pose.
- It converts the heading error into a bounded turn command.
- It keeps forward speed high unless turning saturates.

Important conventions used by the code:

- `x` is left/right
- `y` is forward/back
- `theta` from the robot is reported in degrees, but the controller uses radians
- If `theta` comes from `robotPu.locationArray()[2]`, convert it at the input boundary before using the standard formulas below
- heading uses the standard convention in the XY plane:
 - heading unit vector is `[cos(theta), sin(theta)]`
 - bearing to a point is `atan2(dy, dx)` where `dx = x_target - x`, `dy = y_target - y`

The practical “stop” behavior is:

- Stop when `dist < stopDist`
- `stopDist` is in the same units as odometry (typically mm)

Key tuning knobs from the code:

- `vMax`: maximum forward speed command
- `turnMax`: maximum turn command

- **kTurn**: proportional gain from heading error to turn command (sign depends on robot turning convention)
- **offsetStartDist**: distance at which we start shaping the approach into the final heading
- **leadMin/leadMax**, **lateralOffsetMax**: geometry for the virtual target

Implementation notes (from `robotpu-viewpoint.ts`):

- The controller is intentionally *not* “point straight at the target”. It constructs a **virtual target**
 - slightly past the target pose along `target.theta` (“lead”), and
 - slightly offset left/right of the target heading line (“lateral offset”),
then steers toward that virtual point. This tends to produce smooth approach arcs that arrive already aligned.
- The key computations are:
 - Heading unit vector:


```
* tx = cos(target.theta), ty = sin(target.theta)
```
 - Virtual target point:


```
* xV = target.x + lead*tx + lateral*(-ty)
* yV = target.y + lead*ty + lateral*(tx)
```
 - Bearing to virtual target:


```
* headingToV = atan2(yV - current.y, xV - current.x)
```
 - Heading error and turn command:


```
* eHeading = wrapPi(headingToV - current.theta)
* walkTurn = clamp(kTurn * eHeading, -turnMax, turnMax)
```

Practical usage pattern:

1. Read pose from `robotPu.locationArray()`.
2. Convert the robot’s `theta` (degrees) to radians.
3. Call `updateControl(curPose, targetPose)`.
4. Send commands with `robotPu.walk(walkSpeed, walkTurn)`.

Use Pure Pursuit to follow a list of waypoints

When you have a list of waypoints, “pure pursuit” turns path following into a repeated viewpoint-to-viewpoint problem:

- Maintain a list of waypoints `[P0, P1, ..., Pn]` in world/odometry frame.
- Choose a lookahead distance `L`.
- Find a lookahead point `P*` on the polyline (the point on the path that is $\sim L$ ahead of the robot).

- Set the controller target pose to that lookahead point.

In practice for RobotPU on micro:bit:

- We keep P^* as a position-only target, and set `target.theta` to either:
 - the direction of the next path segment (preferred), or
 - the direction from robot to P^* (simplest)
- We then reuse the same `robotPu.walk(walkSpeed, walkTurn)` loop.

The file `robotpu-pure-pursuit.ts` is a reference implementation that builds on the conventions and controller structure in `robotpu-viewpoint.ts`.

Implementation notes (from `robotpu-pure-pursuit.ts`):

- The path is a waypoint list:
 - `const waypoints: { x: number, y: number }[] = [...]`
- Two key distances control behavior:
 - `LOOKAHEAD_DIST`: how far ahead we “aim” along the polyline.
 - `ARRIVE_DIST`: when we consider a waypoint “reached” and advance `wpIdx`.
- Waypoint advancement logic:
 - If `norm2(waypoints[wpIdx] - curPose) < ARRIVE_DIST`, then `wpIdx += 1`.
- Lookahead target selection (`lookaheadTarget(...)`):
 - Start from the robot position and “walk” along the polyline accumulating segment lengths.
 - When the remaining lookahead distance falls within a segment, interpolate a point on that segment.
 - The returned `target.theta` is set to the segment direction using:
 - * `theta = atan2(dy, dx)` (standard bearing).
- Final control:
 - `target = lookaheadTarget(...)`
 - `controlVec = updateControl(curPose, target)`
 - `robotPu.walk(controlVec[0], controlVec[1])`

Use A* to generate a list of waypoints

If you have a map with obstacles, you can first generate a collision-free *discrete* path using A* on a grid, and then feed the resulting waypoint list into pure pursuit.

Workflow:

1. Represent the field as a grid `grid[i][j]`.
 - 0 means free space
 - 1 means obstacle
2. Run A* from a start cell (`i_start`, `j_start`) to a goal cell (`i_goal`, `j_goal`).
 - Use a simple 4-neighbor model (up/down/left/right)
 - Use Manhattan distance as the heuristic
3. Convert the resulting cell list into *metric* waypoints.
 - Choose a `CELL_SIZE` in your odometry units (usually mm)
 - Convert (`i`, `j`) to waypoint center:
 - `x = (j + 0.5) * CELL_SIZE`
 - `y = (i + 0.5) * CELL_SIZE`
4. Simplify the waypoint list.
 - A* returns many adjacent cells; you typically only need “turning points”.
 - A simple simplifier removes intermediate points that keep the same direction.
5. Paste the simplified waypoint list into `robotpu-pure-pursuit.ts` as `waypoints`.

The reference implementation is in `robotpu-A-star.ts`:

- `astar(grid, start, goal)` returns a `Cell[]` path
- `simplifyPath(path)` reduces it to turning points
- `cellPathToWaypoints(path, CELL_SIZE)` converts to `{x,y}[]`

Implementation notes (from `robotpu-A-star.ts`):

- The implementation is micro:bit friendly:
 - It uses a plain array `open: number[]` and performs a linear scan each iteration to find the best `fScore`.
 - Cells are encoded as an integer key `k = i * cols + j` to avoid allocating many objects.
 - `cameFrom[k]` stores the previous key for path reconstruction.
- Heuristic + neighbors:
 - 4-connected grid (up/down/left/right)
 - Manhattan heuristic: `|di| + |dj|`
- Path simplification:
 - `simplifyPath(...)` keeps only **turning points** by removing intermediate cells that keep the same (`di`, `dj`) direction.

- This produces a much shorter waypoint list that is easier for pure pursuit to follow.

After you have the waypoint list, pure pursuit takes over:

- choose `LOOKAHEAD_DIST`
- follow the waypoint polyline with the same pose conventions as `robotpu-viewpoint.ts`

Advanced background: Divergent Component of Motion (DCM) trajectory generation

This section is background only. You are not expected to implement DCM, MPC, or whole-body control on the micro:bit for the soccer final project.

Instead of planning a static 2D line, the gold standard for time-optimal bipedal locomotion is generating a trajectory based on the **Divergent Component of Motion (DCM)** (also frequently called the **Capture Point**).

- **How it works:** The DCM splits the complex, unstable physics of Robot PU's walking balance into a stable component and an unstable (divergent) component. The unstable component describes exactly how the robot's momentum is "falling" forward.
- **Why it solves your problem:** To get from A to B as fast as possible, the DCM algorithm calculates an aggressive forward-falling trajectory. It figures out the absolute minimum number of footsteps required to throw the robot's center of mass (CoM) toward Point B, accelerating immediately to `max_move_speed` and calculating the exact foot placement angles to arrest the momentum perfectly into the final `heading_2`.

The Execution Engine: Model Predictive Control (MPC) with a Receding Horizon

To actually calculate this path in real-time, you should utilize **Model Predictive Control (MPC)** based on a **Linear Inverted Pendulum Model (LIPM)**.

- **How it works:** The MPC looks ahead over a short horizon (typically the next 2 to 4 footsteps). It sets up an optimization problem (Quadratic Programming) where the objective function is to **minimize time/maximize velocity** toward Pose B.
- **Handling the Constraints:** You feed your `max_move_speed` and `max_turn_speed` directly into the MPC as hard mathematical boundaries. The algorithm then solves for the fastest sequence of individual foot step matrices $(x, y, \theta)_{step}$ and duration per step (t_{step}) to transition from Pose 1 to Pose 2.

- **The Result:** If Pose B requires a sharp heading change, the MPC won't make the robot stop and turn; it will plan an optimized “banking” or curving run—similar to an athlete running a sprint around a track—spinning the torso concurrently while moving laterally.

How to Practicalize It (The Software Pipeline)

If you don't want to code the raw predictive physics matrices from scratch, you can split the problem into a high-level time-optimal path generator and a low-level footstep compiler:

Step 1: High-Level Path (Quintic Hermite Spline)

Generate a 2D geometric path using a **Quintic Hermite Spline** between A and B. Unlike cubic splines, quintic splines allow you to constrain position, velocity, and acceleration at both start and endpoints. This ensures that your initial and final velocities match your required headings flawlessly.

Step 2: Time Parameterization (Trapezoidal / S-Curve Profile)

Apply a **Trapezoidal Motion Profile** across the generated spline. This profile scales the timeline so that Robot PU is pinned exactly at its `max_move_speed` for the maximum possible duration of the journey, calculating the exact millisecond it needs to begin decelerating to stop cleanly at Point B.

Step 3: Feeding Stanley/Pure Pursuit Alternatives into Whole-Body Control (WBC)

Instead of feeding this path into Stanley, you pass this time-parameterized spline into a **Footstep Planner (like the DCM or LIPM MPC)**. The planner acts as the “driver,” translating the optimal 2D trajectory into discrete left-and-right foot placements. These are then executed by a **Whole-Body Controller (WBC)** which regulates the joint torques to ensure the robot doesn't slip or tip over.

Summary Recommendation

To achieve the absolute fastest run, look into the open-source library **OCS2 (Optimal Control for Switched Systems)** or **Crocoddyl** (Contact-Consistent Trajectory Optimization). These frameworks are specifically designed for legged robots; they take your start pose, end pose, and velocity limits, and automatically synthesize a time-optimal, physically stable walking trajectory.

Chapter 5: Sensor Fusion

In this chapter, we first build a micro:bit-feasible fusion pipeline (gating + prediction + smoothing), then show how a small Kalman filter improves the stability of the kick-point planner.

Code reference for this chapter:

- `robotpu-odometry.ts`: Odometry for prediction (pose propagation)
- `robotpu-soccer-mvp.ts`: end-to-end MVP loop (camera packet -> local frame -> kick point -> walking)
- `robotpu-kalman-filter.ts`: MVP loop with Kalman filters added for ball/goal smoothing
- `robotpu-soccer-demo.ts`: final integrated demo that combines fusion with planning and control

1. What are we estimating?

In this project we represent the “field” as a small 10×10 grid (1 m x 1 m). The most useful states to estimate on micro:bit are:

- **Robot pose in odometry frame:** (x, y, θ) for latency compensation.
- **Ball position in the current local frame** C_{now} : (x_{ball}, y_{ball}) .
 - Before the kick: ball is mostly stationary.
 - After the kick: ball can move, so a constant-velocity model helps.
- **Goal position in the current local frame** C_{now} : (x_{goal}, y_{goal}) .
 - The goal is physically stationary, but camera estimates jitter and can drop out.
 - Filtering is still helpful because heading alignment uses the goal bearing.
- **Obstacle layer** (optional): a small occupancy belief grid from sonar.

2. Fusion pattern: prediction + correction

Micro:bit friendly fusion follows a standard pattern:

- **Prediction:** propagate your state forward using a motion model.
 - For object tracking (ball/goal): predict based on a simple constant-position or constant-velocity model.
 - For robot pose: use odometry (see `02_odometry.md`). Use standard math convention internally and convert RobotPU-reported heading when needed.
- **Correction:** when a new camera packet arrives and is trustworthy, update the estimate.

3. Camera measurement robustness (must-have)

- Use **VALID** / **STALE** to gate updates
- Reject impossible jumps
- Smooth ball/goal estimates (low-pass filter) and keep an age counter

In the MVP programs, the **VALID**/**STALE** flags are already used to avoid overwriting state with bad measurements. A Kalman filter builds directly on the same idea:

- If a measurement is not **VALID**: do **predict-only**.
- If a measurement is **STALE**: either do predict-only, or update with a **larger measurement noise** (trust it less).

4. Micro:bit fusion recipe (recommended)

The MVP loop (`robotpu-soccer-mvp.ts`) already uses a practical structure:

1. **Read and gate camera packets** using **VALID**/**STALE**.
2. **Store measurement at receipt time** plus odometry pose at that time.
3. **Latency compensate** into the current local frame using `camToNow(...)`.
4. **Filter** the resulting `ball_now` and `goal_now`.
5. **Plan in the current local frame**:
 - compute `kickPt = computeKickPoint(ball, goal)`
 - compute `thetaKick` (face goal at kick point)
 - drive to `kickPt` using `updateControl(...)`

5. Sonar fusion for mapping

- Update an obstacle belief grid (log-odds integers)
- Optional: free-space carving
- Add time decay so false positives do not persist forever

6. Kalman filter role in this fusion

Kalman filters are most useful here as *measurement smoothers* for the ball and goal positions used by planning.

In `robotpu-kalman-filter.ts` we use a micro:bit-feasible Kalman design:

- Per axis 1D constant-velocity state

$$x = \begin{bmatrix} p \\ v \end{bmatrix}$$

- Transition model for timestep Δt (seconds)

$$F = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}$$

- Measurement is position only

$$z = Hx + \epsilon, \quad H = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

- Process noise Q and measurement noise R are treated as simple scalars per axis for implementability.

We run two independent filters (x/y) for each target, which is equivalent to a 2D filter with diagonal covariance.

6.1 Goal filter (stationary target)

Even though the goal does not move, the measured `goal_now` will jitter. Filtering reduces oscillation in:

- goal alignment (`headingGoal`)
- `thetaKick` computed from (`ball`, `goal`) geometry

Recommended model:

- State per axis: $[p, v]$ (position + velocity)
- Use very small process noise so the filter behaves like a “sticky” average.

Implementation detail that exploits “goal does not move”:

- Keep Q extremely small (the filter will *resist* drift)
- Still allow some Q on velocity so the filter can absorb occasional small apparent motion from camera jitter
- When goal measurements drop out, the predict step alone holds a stable estimate for planning

6.2 Ball filter (stationary before kick, moving after)

- Before kick: treat the ball as approximately stationary (small process noise).
- After kick: the ball may move, so allow more process noise (or keep constant-velocity model with higher process noise).

In `robotpu-kalman-filter.ts` we explicitly switch the ball process noise after triggering a kick:

- Before kick:
 - Small Q (ball assumed nearly fixed in the local frame)
- After kick for a short window (`BALL_POSTKICK_MS`):

- Larger Q so the filter can quickly follow the ball if it rolls
- This reduces “lag” compared to a low-pass filter

This simple mode switch matches the soccer task structure and avoids keeping a high-noise model all the time.

If you want the filter to re-freeze after the kick, you can reset `lastKickMs` or reduce the post-kick window.

6.3 Why this helps the MVP controller

The MVP controller computes `kickPt` from `ball_now` and `goal_now`. If either measurement jitters, then:

- `kickPt` jumps
- `thetaKick` jumps
- `walkTurn` oscillates (zig-zag walking)

Filtering makes the controller’s target pose stable enough for a biped gait.

7. How the code maps to the math

`robotpu-kalman-filter.ts` integrates filtering *after* latency compensation:

1. Build measurements in the current frame:
 - `ball_meas = camToNow(ball_cam2D, ball_pose_0, pose_now_0)`
 - `goal_meas = camToNow(goal_cam2D, goal_pose_0, pose_now_0)`
2. Predict both filters using `dt_s = (now_ms - lastKfMs)/1000`:
 - `goalkF.predict(dt_s, GOAL_Q_POS, GOAL_Q_VEL)`
 - `ballKF.predict(dt_s, bq_pos, bq_vel)`
3. Update only when a fresh measurement is available:
 - `goalkF.update(goal_meas[0], goal_meas[1], GOAL_R_FRESH)`
 - `ballKF.update(ball_meas[0], ball_meas[1], BALL_R_FRESH)`
4. Planner uses filtered state:
 - `ball_now = ballKF.pos()`
 - `goal_now = goalkF.pos()`

This ordering matters:

- `camToNow(...)` corrects for robot motion during camera latency
- Kalman filter then smooths camera jitter *in the current planning frame*

8. Gating and dropout behavior

- If a packet is not `VALID` it is treated as “no measurement”. The filter runs prediction only.

- This repo’s MVP loop also drops measurements that are too old (`LOST_TIMEOUT_MS`). When that happens the planner ignores that target.

Recommended extension (optional): when `STALE`, still call `update(...)` but with a larger measurement noise (e.g. `GOAL_R_STALE`, `BALL_R_STALE`). In the current code we simply do not store stale measurements into `*_cam2D`, so stale updates do not occur.

9. Practical tuning tips (micro:bit scale)

- R (measurement noise): increase if the robot “chases jitter”
- Q (process noise): increase if the estimate lags behind real motion
- Goal tuning:
 - Prefer tiny `GOAL_Q_POS` so the goal estimate is stable even with dropouts
- Ball tuning:
 - Keep `BALL_Q*_PRE` small for stable approach
 - Use larger `BALL_Q*_POST` so the estimate can follow rolling after kick

10. Checklist and test protocol

- Verify signs and axes for camera-to-robot transform
- Verify `VALID/STALE` behavior (dropouts should not cause jumps)
- Replay fixed step sequences to confirm drift correction improves stability

Chapter 6: Soccer MVP

This chapter packages the work from Chapters **02–05** into a single end-to-end demo program that performs the basic robot soccer behavior:

1. Detect ball and goal
2. Latency-compensate detections into the current local frame using odometry
3. Smooth ball/goal with a small Kalman filter
4. Compute a kick point behind the ball
5. Plan toward the kick point on a small grid (A^*)
6. Use a heading-aware controller to approach, align, and kick

Code reference for this chapter:

- `robotpu-soccer-demo.ts`

1. Inputs and outputs

Inputs:

- **Odometry pose** from `robotPu.locationArray()`
 - Interpreted as $[x, y, \theta]$ in an odometry frame O
 - Used only to compensate camera latency (no global localization required)
- **Camera detections** from the ESP32 module (I2C packets)
 - Ball and goal detections include **VALID** and **STALE**
 - Detections arrive with non-trivial latency and jitter

Outputs:

- `robotPu.walk(walkSpeed, walkTurn)`
- the RobotPU kick action, such as `robotPu.kick()` if available in your RobotPU API, or a short forward push behavior

2. Coordinate and frame conventions (what the demo actually uses)

This demo plans in a **current local frame** anchored at “now”:

- The robot is always at $(0, 0)$ in the planning frame.
- The ball and goal are represented as 2D points $[x, y]$ in meters in that current local frame.

The camera packet values are stored at receipt time together with the odometry pose at that moment, then transformed into the current local frame using odometry.

Angle convention note:

- Internally, the demo uses the standard convention: `theta=0` along $+X$, positive CCW toward $+Y$, bearing `atan2(dy, dx)`.
- RobotPU reports `theta_robot_deg` with 0 meaning $+Y$ forward, so convert at the input boundary:
 - `theta_std_deg = 90 - theta_robot_deg`
 - `theta_std_rad = deg2rad(theta_std_deg)`

3. End-to-end pipeline in `robotpu-soccer-demo.ts`

3.1 Packet gating and measurement storage

The demo reads one I2C packet per control loop:

- If **VALID** is not set: treat as no detection.

- If **STALE** is set: the measurement is not stored (the program preserves the last good one).

When a fresh measurement arrives it stores:

- `ball_cam2D` / `goal_cam2D` (meters, ground-plane projected)
- `ball_pose_0` / `goal_pose_0` (odometry pose at receipt)
- `ball_rx_ms` / `goal_rx_ms` (timestamp)

3.2 Latency compensation into the current frame

Each cycle, the demo computes:

- `ball_meas = camToNow(ball_cam2D, ball_pose_0, pose_now_0)`
- `goal_meas = camToNow(goal_cam2D, goal_pose_0, pose_now_0)`

This uses odometry to correct for the robot's motion between receipt time and now.

3.3 Kalman smoothing (sensor fusion for planning stability)

After latency compensation, the demo filters ball and goal with lightweight constant-velocity Kalman filters (per axis $[p, v]$).

Two task-specific behaviors are implemented:

- **Goal is stationary:**
 - very small process noise (`GOAL_Q_POS`, `GOAL_Q_VEL`)
 - goal estimate stays stable even if a few frames drop
- **Ball changes after kick:**
 - before kick: small process noise (`BALL_Q_POS_PRE`, `BALL_Q_VEL_PRE`)
 - for a short window after the kick action (`BALL_POSTKICK_MS`): larger process noise (`BALL_Q_POS_POST`, `BALL_Q_VEL_POST`)

The planner uses the filtered positions `ball_now` and `goal_now`.

3.4 Kick point computation

The demo does not navigate “to the ball”. It navigates to a **kick point** behind the ball along the goal→ball direction:

- `kickPt = computeKickPoint(ball_now, goal_now)`

This makes it more likely the ball is pushed toward the goal instead of sideways.

4. Planning: A* on the 10x10 local grid

The demo uses a tiny robot-local planning grid and A* because it is micro:bit-feasible. This local planning grid is not necessarily the entire 1 m x 1 m game field:

- **Grid:** GRID_N = 10, GRID_RES_M = 0.05
- **Neighbors:** 4-connected
- **Heuristic:** Manhattan distance

Implementation usage:

- Plan from the robot's cell (treated as (0,0) in the current frame) to the kick point cell.
- Execute **only the first step** of the planned path.
- Re-plan every cycle.

Current alpha constraint:

- The occupancy grid passed to A* marks the **ball cell and goal cell as obstacles** (this is temporary and only for demonstration).

5. Control: heading-aware approach, then align + kick

The program uses a heading-aware controller `updateControl(current, target)`:

- The current pose is always { `x: 0, y: 0, theta: 0` } because we are in the current local frame.
- The target pose is the next A* step center (or the kick point if no path exists).

End-heading logic:

- When far from the kick point, the target heading points toward the next step.
- When near the kick point, the target heading switches to `thetaKick` (the desired kick direction).

Kick behavior:

- If close enough to kick point (`KICK_DIST_M`), the demo aligns to face the goal direction.
- If heading error is small, it triggers the RobotPU kick action, such as `robotPu.kick()` if available in your RobotPU API, or a short forward push behavior.

6. Search behavior

If the ball is not currently valid, the demo stops walking and runs a head scanning pattern (`SEARCH_PATTERN`) to reacquire the ball.

7. Known limitations (alpha)

- Sonar obstacles are not integrated in `robotpu-soccer-demo.ts` yet.

- This chapter assumes a future sonar API for immediate collision avoidance.
- Right now the “obstacle avoidance” behavior is limited to the A* grid occupancy used in the demo.
- **Ball/goal treated as obstacles in A*.**
 - This is intentionally simplistic and may block paths near the kick point.
 - The recommended next step is to treat only real obstacles as occupied and keep ball/goal as tracked targets.

8. Suggested testing protocol

- Confirm camera packet parsing and **VALID/STALE** gating (no jumps on dropouts)
- Confirm `camToNow(...)` sign conventions by moving the robot and watching `ball_now` stability
- Tune Kalman noise values:
 - increase `*_R_*` if you see jitter chasing
 - increase `BALL_Q*_POST` if the filtered ball lags after kick
- Validate the A* loop:
 - print path length and next step choice
 - verify replanning is stable as ball/goal move
- Validate kick behavior:
 - check kick point geometry
 - check final alignment threshold before kicking

Chapter 7: Behavior Tree Planner

This chapter replaces the flat `if/else` planner from Chapter 6 (`06_soccer_mvp.md`) with a proper **Behavior Tree (BT)**. The same logic is restructured into explicit **Sequence** and **Fallback** nodes so the control flow is readable, testable, and easy to extend.

Code reference for this chapter:

- `robotpu-behavior-tree.ts` (or merge into `robotpu-soccer-mvp.ts`)

Prerequisites: read Chapter 6 first — this chapter assumes familiarity with `ball_valid`, `goal_valid`, `ball_now`, `goal_now`, `walkMode`, `computeKickPoint`, `updateControl`, `astar`, and the actuator loop.

Tree structure

```
Root (Fallback)
  Score (Sequence)
    COND ballVisible()          -- ball freshness check
    COND goalVisible()          -- goal freshness check
    COND pathClear()            -- no obstacle between robot and kick point
    ACTION approachKickPoint()  -- walk to kick point (RUNNING / SUCCESS)
    ACTION alignToGoal()        -- turn to face goal (RUNNING / SUCCESS)
    ACTION kick()               -- execute kick motion (SUCCESS once)
  ApproachBallOnly (Sequence)
    COND ballVisible()
    ACTION approachBall()       -- ball seen but no goal: chase ball
  ACTION searchBallBT()        -- nothing seen: scan with head
```

Tick semantics used here: - SUCCESS (2) — condition is true or action completed - FAILURE (1) — condition is false or action cannot run - RUNNING (0) — action is in progress, re-tick next cycle

Full TypeScript implementation

Paste this file into MakeCode as a new file, or merge into `robotpu-soccer-mvp.ts` replacing the `if (ball_valid && goal_valid) { ... }` planner block.

```
//
// BT node return values
//
const BT_RUNNING = 0
const BT_FAILURE = 1
const BT_SUCCESS = 2

//
// BT composite helpers
//

// Sequence: tick children left to right; return FAILURE on first FAILURE,
// return RUNNING on first RUNNING, return SUCCESS only if all succeed.
function btSequence(children: (() => number)[]): number {
  for (let i = 0; i < children.length; i++) {
    const result = children[i]()
    if (result !== BT_SUCCESS) return result // FAILURE or RUNNING stops here
  }
  return BT_SUCCESS
}

// Fallback (Selector): tick children left to right; return SUCCESS on first
```

```

// SUCCESS, return RUNNING on first RUNNING, return FAILURE only if all fail.
function btFallback(children: (() => number)[]): number {
    for (let i = 0; i < children.length; i++) {
        const result = children[i]()
        if (result !== BT_FAILURE) return result // SUCCESS or RUNNING stops here
    }
    return BT_FAILURE
}

//
// Condition nodes (instant, no side-effects, return SUCCESS or FAILURE only)
//

// Is the ball measurement fresh enough to trust?
function condBallVisible(): number {
    const age = input.runningTime() - ball_rx_ms
    return (ball_valid && age < LOST_TIMEOUT_MS) ? BT_SUCCESS : BT_FAILURE
}

// Is the goal measurement fresh enough to trust?
function condGoalVisible(): number {
    const age = input.runningTime() - goal_rx_ms
    return (goal_valid && age < LOST_TIMEOUT_MS) ? BT_SUCCESS : BT_FAILURE
}

// Is the straight-line path to the kick point obstacle-free?
// Uses the same LocalGrid that the main loop fills every cycle.
// A cell value >= 1 means occupied. We check the 3 cells directly ahead.
function condPathClear(): number {
    if (!ball_valid || !goal_valid) return BT_FAILURE
    const kickPt = computeKickPoint(ball_now, goal_now)
    // Sample 3 waypoints between robot (0,0) and kick point
    for (let t = 1; t <= 3; t++) {
        const fx = kickPt[0] * t / 4
        const fy = kickPt[1] * t / 4
        const cell = grid.index(fx, fy)
        if (grid.get(cell[0], cell[1]) >= 1) return BT_FAILURE
    }
    return BT_SUCCESS
}

// Is the robot already at (or past) the kick point?
function condAtKickPoint(): number {
    if (!ball_valid || !goal_valid) return BT_FAILURE
    const kickPt = computeKickPoint(ball_now, goal_now)
    return norm2L(kickPt[0], kickPt[1]) <= KICK_DIST_M ? BT_SUCCESS : BT_FAILURE
}

```

```

}

// Is the robot heading aligned to the goal (within 0.25 rad ~ 14 deg)?
function condAlignedToGoal(): number {
  if (!goal_valid) return BT_FAILURE
  const heading = desiredHeadingTo(goal_now[0], goal_now[1])
  return Math.abs(heading) <= 0.25 ? BT_SUCCESS : BT_FAILURE
}

//
// Action nodes (may return RUNNING across ticks; set walkSpeed/walkTurn/walkMode)
//

// Walk toward the kick point. Returns RUNNING until close enough.
function actionApproachKickPoint(): number {
  if (!ball_valid || !goal_valid) return BT_FAILURE
  const kickPt = computeKickPoint(ball_now, goal_now)
  const dist = norm2L(kickPt[0], kickPt[1])
  if (dist <= KICK_DIST_M) return BT_SUCCESS

  const thetaKick = desiredHeadingTo(
    goal_now[0] - kickPt[0],
    goal_now[1] - kickPt[1]
  )
  const ctrl = updateControl(
    { x: 0, y: 0, theta: 0 },
    { x: kickPt[0], y: kickPt[1], theta: thetaKick },
    0.25,
    KICK_DIST_M
  )
  walkSpeed = ctrl[0]
  walkTurn = ctrl[1]
  walkMode = 0
  return BT_RUNNING
}

// Rotate in place to face the goal. Returns RUNNING until aligned.
function actionAlignToGoal(): number {
  if (!goal_valid) return BT_FAILURE
  const heading = desiredHeadingTo(goal_now[0], goal_now[1])
  walkTurn = clampL(TURN_GAIN * heading, -0.8, 0.8)
  if (Math.abs(heading) > 0.25) {
    walkSpeed = 1.0
    walkMode = 1 // back-up-and-turn mode
  }
  return BT_RUNNING
}

```

```

    walkSpeed = 0
    walkMode = 0
    return BT_SUCCESS
}

// Execute the kick. Sets walkMode = 2 for one cycle, then returns SUCCESS.
function actionKick(): number {
    walkSpeed = 0
    walkTurn = 0
    walkMode = 2 // kick mode; picked up by the actuator loop
    return BT_SUCCESS
}

// Approach the ball when no goal is visible (fallback chase).
function actionApproachBall(): number {
    if (!ball_valid) return BT_FAILURE
    const dist = norm2L(ball_now[0], ball_now[1])
    const heading = desiredHeadingTo(ball_now[0], ball_now[1])
    walkTurn = clampL(TURN_GAIN * heading, -0.8, 0.8)
    walkSpeed = dist > 0.25 ? 2.0 : 1.0
    walkMode = 0
    return BT_RUNNING
}

// Sweep head to find the ball (no ball seen at all).
function actionSearchBallBT(): number {
    walkSpeed = 0
    walkTurn = 0
    walkMode = 0
    searchBall(SEARCH_PATTERN)
    return BT_RUNNING // always running; an external event (ball detected) breaks out
}

//
// Replan node: recalculate path via A* and return first waypoint.
// Inserted between condPathClear (when it returns FAILURE) and approach.
//
function actionReplan(): number {
    if (!ball_valid || !goal_valid) return BT_FAILURE
    const kickPt = computeKickPoint(ball_now, goal_now)

    const occ = new LocalGrid()
    occ.clear()
    // Mark ball and goal cells as obstacles for path planning
    const b = occ.index(ball_now[0], ball_now[1])
    occ.set(b[0], b[1], 1)

```

```

const g = occ.index(goal_now[0], goal_now[1])
occ.set(g[0], g[1], 1)

const start = occ.index(0, 0)
const goalCell = occ.index(kickPt[0], kickPt[1])
const path = astar(start[0], start[1], goalCell[0], goalCell[1], occ)

if (path.length < 2) return BT_FAILURE // no path found

// Step toward first waypoint along the found path
const next = path[1]
const ni = (next / GRID_N) | 0
const nj = next % GRID_N
const wp = occ.center(ni, nj)

const thetaKick = desiredHeadingTo(
  goal_now[0] - kickPt[0],
  goal_now[1] - kickPt[1]
)
const ctrl = updateControl(
  { x: 0, y: 0, theta: 0 },
  { x: wp[0], y: wp[1], theta: thetaKick },
  0.50,
  KICK_DIST_M
)
walkSpeed = ctrl[0]
walkTurn = ctrl[1]
walkMode = 0
return BT_RUNNING
}

//
// Root tree definition
//

// Navigate-or-replan sub-tree:
// Try direct approach first; if path is blocked, replan via A*.
function btNavigateToKick(): number {
  return btFallback([
    // Direct approach: only if path is clear
    () => btSequence([condPathClear, actionApproachKickPoint]),
    // Replan via A* when direct path is blocked
    actionReplan
  ])
}

```

```

// Full score sequence: see ball + goal, navigate, align, kick.
function btScore(): number {
    return btSequence([
        condBallVisible,
        condGoalVisible,
        btNavigateToKick,
        actionAlignToGoal,
        actionKick
    ])
}

// Ball-only fallback: chase ball when goal not yet visible.
function btApproachBallOnly(): number {
    return btSequence([
        condBallVisible,
        actionApproachBall
    ])
}

// Root: try to score, else chase ball, else search.
function btRoot(): number {
    return btFallback([
        btScore,
        btApproachBallOnly,
        actionSearchBallBT
    ])
}

//
// Integration: call btRoot() in the main planning loop
// Replace the if/else planner block in robotpu-soccer-mvp.ts with:
//
//   btRoot()
//
// The actuator loop remains unchanged:
//
//   basic.forever(function () {
//       if (walkMode == 0) { robotPu.walk(walkSpeed, walkTurn) }
//       else if (walkMode == 1) { robotPu.walk(-walkSpeed, walkTurn) }
//       else if (walkMode == 2) { robotPu.kick() }
//       basic.pause(10)
//   })
//

```

How to integrate with robotpu-soccer-mvp.ts

In the main `basic.forever` planning loop, replace:

```
// OLD flat if/else planner (lines 499-548 in robotpu-soccer-mvp.ts)
if (ball_valid && goal_valid) {
    ...
} else if (ball_valid) {
    ...
} else {
    ...
}
```

with:

```
// NEW: single BT tick per cycle
btRoot()
```

The `ball_now` and `goal_now` variables (computed from `camToNow`) must be in scope before `btRoot()` is called — they already are in the existing loop.

Extending the tree

Goal	Where to add
Recover from fall	New <code>condRobotFallen</code> + <code>actionStandUp</code> as first child of root Fallback
Avoid a moving opponent	New <code>condOpponentBlocking</code> + <code>actionSidestep</code> inside <code>btNavigateToKick</code>
Use Kalman-smoothed positions	Swap <code>ball_now</code> / <code>goal_now</code> for <code>ballKF.pos()</code> / <code>goalKF.pos()</code> (already in <code>robotpu-soccer-demo.ts</code>)
Add goalie mode	New top-level Fallback child <code>btDefend()</code> triggered by ball approaching own goal

Tick trace example

Node	Tick A: searching	Tick B: ball seen, no goal	Tick C: ball+goal, navigating	Tick D: at kick point, aligned
Root	RUNNING	RUNNING	RUNNING	SUCCESS
Fallback				
btScore	FAILURE	FAILURE	RUNNING	SUCCESS
condBallVisible	FAILURE	SUCCESS	SUCCESS	SUCCESS
condGoalVisible	—	FAILURE	SUCCESS	SUCCESS
btNavigateToKick	—	—	RUNNING	SUCCESS
actionAlignToGoal	—	—	—	SUCCESS
actionKick	—	—	—	SUCCESS
btApproachBall	FAILURE	RUNNING	—	—
actionSearchBall	RUNNING	—	—	—